

GESTIÓN DE ARCHIVOS

SISTEMAS OPERATIVOS

DEPARTAMENTO CIENCIAS DE LA COMPUTACIÓN

FACYT-UC

PROF.PhD. MIRELLA HERRERA

Almacenamiento Secundario

Función principal

- El SA establece correspondencia entre los archivos y dispositivos lógicos

Características para el usuario

- Almacenamiento permanente de información. No desaparecen aunque se apague el computador
- Conjunto de información estructurada de forma lógica según criterios de aplicación
- Nombres lógicos y estructurados
- No están ligados al ciclo de vida de una aplicación particular
- Abstraen los dispositivos físicos
- Se acceden a través de llamadas al sistema operativo o bibliotecas de utilidades

Almacenamiento Secundario

El acceso a los dispositivos es:

- Incomodo
- Detalles físicos de los dispositivos
- Dependiente de las direcciones físicas
- No seguro
- Si el usuario accede a nivel físico no tiene restricciones

El sistema de archivos es la capa de software entre dispositivos y usuarios

Objetivos:

- Suministrar una visión lógica de los dispositivos
- Ofrecer primitivas de acceso cómodas e independientes de los detalles físicos
- Mecanismos de protección

Almacenamiento Secundario

Características físicas:

- Barato (en comparación con la RAM)
- Lento (en comparación con la RAM)
- No volátil
- Heterogéneo (cintas y discos magnéticos, discos ópticos, discos SSD, tarjetas extraíbles...)

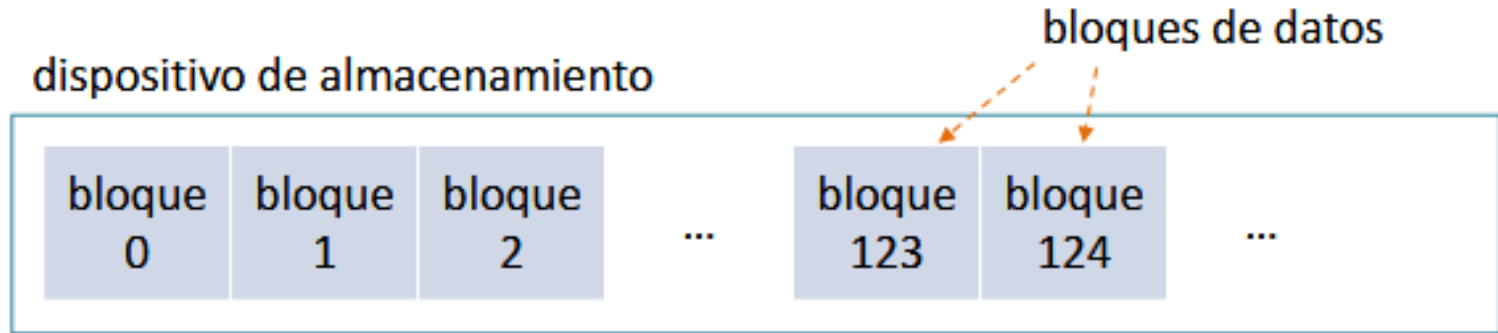
En un sistema típico, el almacenamiento secundario se utiliza para:

- Almacenar el código ejecutable de las aplicaciones
- Almacenar los documentos de los usuarios
- Guardar copias de seguridad
- Servir de respaldo para la memoria virtual

Almacenamiento Secundario

El núcleo ofrece una interfaz abstracta para trabajar con la E/S de almacenamiento.

Un modelo típico es abstraer el dispositivo como un *array* de bloques de datos de tamaño fijo, ej. `write_block(block_id, data[])`



Almacenamiento Secundario

La interfaz de E/S del núcleo tiene un nivel de abstracción demasiado bajo para las necesidades y expectativas de los usuarios.

Los usuarios trabajan con este tipo de entidades:

- Aplicaciones (programas ejecutables)
- Documentos de texto
- Bases de datos
- Fotos, vídeos, música

A un usuario (o a un programador) no podemos decirle que para leer un documento tenga que ordenarle al sistema: “busca en el disco los bloques 357, 48 y 13”

Concepto de Archivo

El sistema operativo ofrece una entidad abstracta más próxima a los conceptos que maneja el usuario: ARCHIVO / FICHERO / file

- Un archivo es un objeto identificable mediante un nombre, que guarda información persistente. Ejemplos: “mifoto.jpg”, “apuntes-fso.docx”, “winword.exe”
- Persistente = no volátil = duradera
- Sobre un archivo se pueden realizar diversas operaciones: crear, leer, escribir, eliminar, etc.
- Un archivo es un tipo abstracto de datos = operaciones públicas + estructura de datos

Atributos del Archivo

Además del contenido, cada archivo tiene unos atributos que describen sus características:

- Nombre (cadena de caracteres)
- Tipo de archivo (necesario en sistemas que reconocen distintos tipos)
- Localización en el dispositivo de almacenamiento
- Tamaño
- Información de protección: permisos, usuarios autorizados, etc.
- Fechas y horas de creación, modificación, último acceso, etc.
- Versión (en sistemas que manejan historial de versiones)

Cada SO establece un conjunto estándar de atributos para sus archivos, y limitaciones sobre ellos (ej. longitud máxima del nombre)

Tipos de archivo

La información guardada puede ser de muchos tipos: ejecutables, textos, fotos, bases de datos...

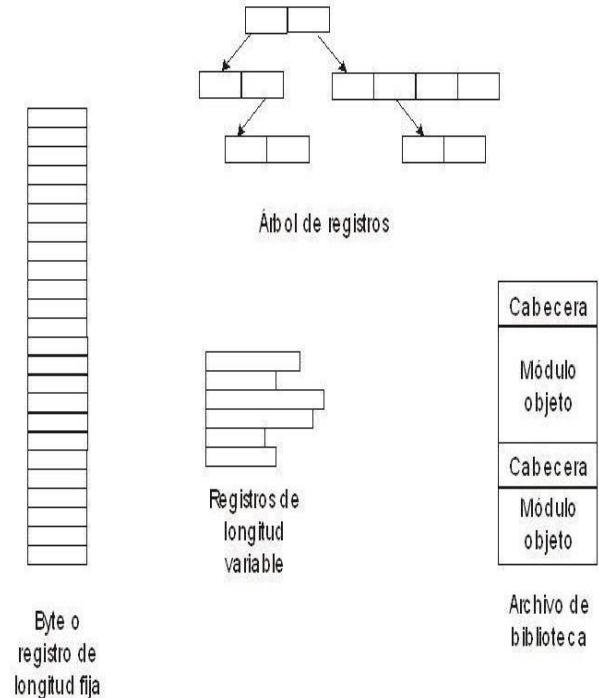
¿Cómo reconoce el SO qué tipo de información contiene el archivo?

- En Windows se usan las últimas letras del nombre como indicador del tipo
- En Mac (versiones antiguas) unos atributos marcan el tipo de archivo
- En Unix el SO no reconoce tipos de archivos

Todos los S.O. deben reconocer al menos un formato de archivo: la de un archivo ejecutable

Estructura de Archivo

- Ninguna – secuencia de palabras o bytes (UNIX)
- Estructura sencilla de registros
 - Líneal
 - Longitud fija
 - Longitud variable
- Estructuras complejas
- Documentos con formato (HTML, postscript)
- Archivo de carga reubicable (módulo de carga)
- Se puede simular estructuras de registro complejas con una estructura plana y secuencias de control
- ¿Quién decide la estructura?
 - Interna: El sistema operativo
 - Externa: Las aplicaciones



Gestión interna de archivo

Para dar soporte a la traducción entre la visión “ristra de bytes” y el almacenamiento real “bloques de disco” debe:

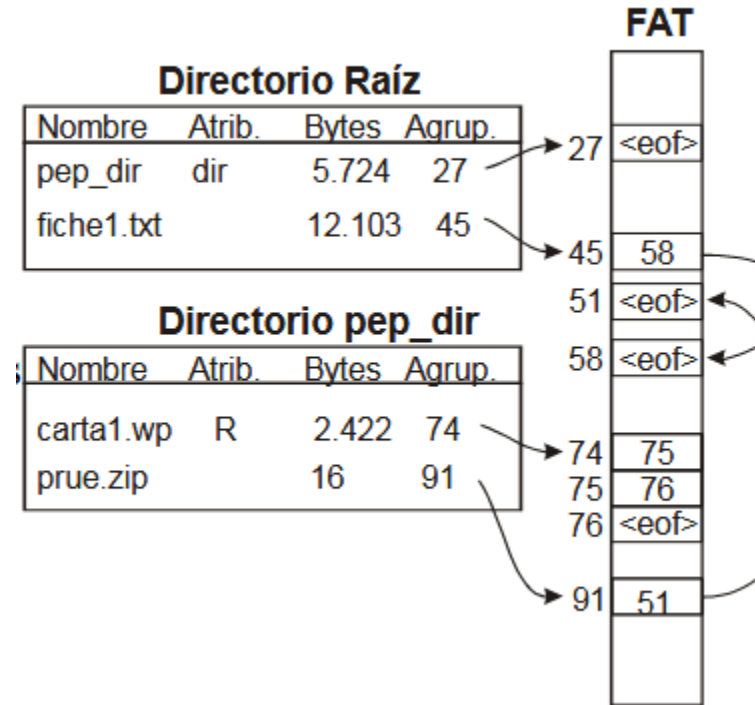
- Haber una estructura (interna al SA) que represente qué bloques de disco pertenecen a un archivo
- Esta estructura se tiene que almacenar (exactamente igual que se almacenan los datos)
- Asociada a esta estructura se guarda más meta-información del archivo
 - ISO9660: Inicio y tamaño (fichero contiguo)
 - SF MS-DOS: FAT (fichero enlazado)
 - SF UNIX: i-nodo (fichero indexado)
 - NTFS: Registro Windows (fichero indexado)

Gestión interna de archivo

Archivos enlazados FAT (File Allocation Table) – MS DOS

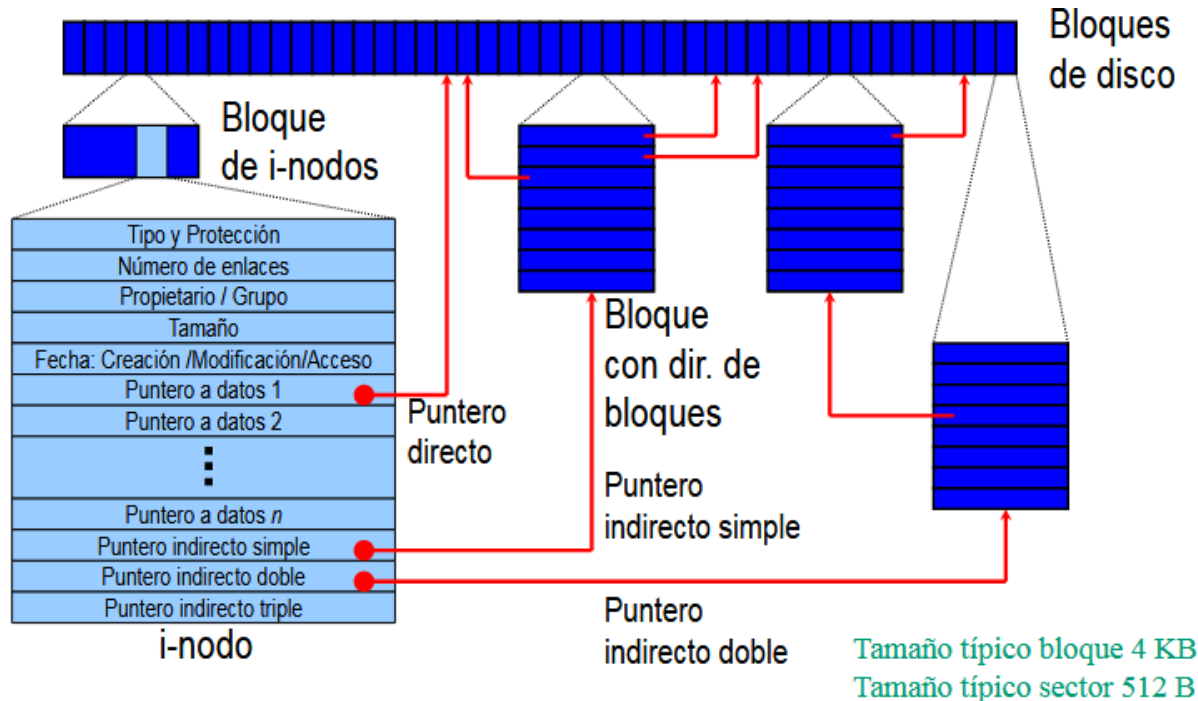
Primer byte del nombre:

- 0x00 entrada libre
- 0xE5 entrada borrada
- 0x05 primer carácter es E5
- FAT de 12 bits 4K agrupaciones
- FAT de 16 bits 64K agrupaciones
- FAT de 32 bits 228 agrupaciones (solo usa 28 bits)
- Atributos: System, Hidden, Read-only, Archived next backed, Directory, Volume



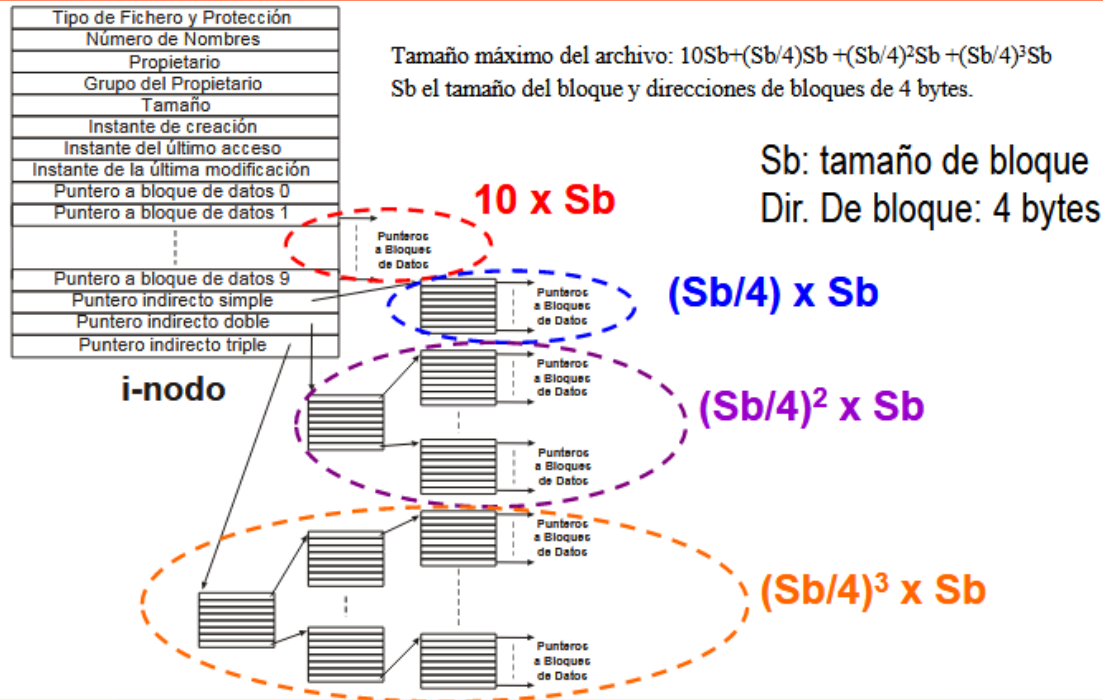
Gestión interna de archivo

Archivos indexados: Nodos i - UNIX



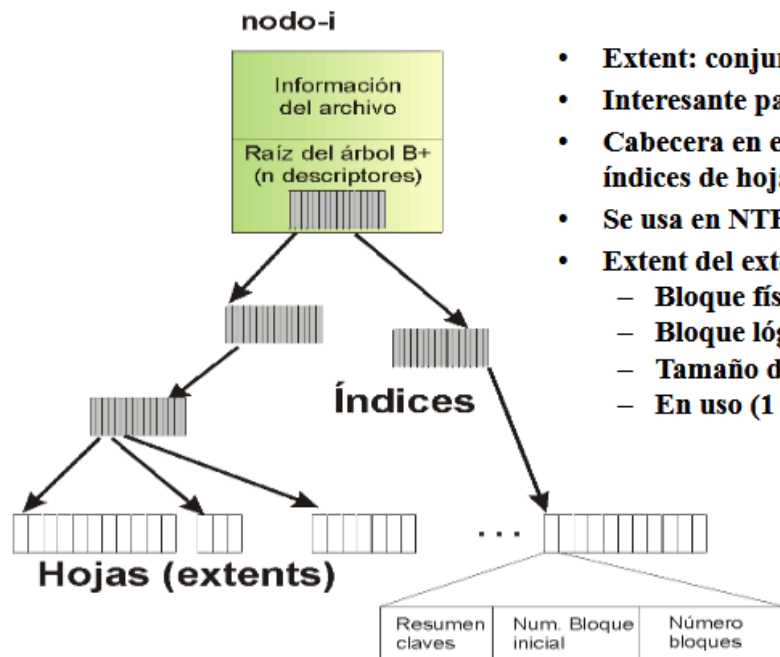
Gestión interna de archivo

Archivos indexados: Nodos i - UNIX



Gestión interna de archivo

Árboles balanceados: Extent (ext...) - NTFS



- **Extent: conjunto contiguo de bloques**
- **Interesante para grandes ficheros contiguos**
- **Cabecera en el bloque para diferenciar índices de hojas (extents)**
- **Se usa en NTFS y ext4**
- **Extent del ext4 (96 bits):**
 - **Bloque físico (48 bits)**
 - **Bloque lógico (32 bits)**
 - **Tamaño del extent en bloques (15 bits)**
 - **En uso (1 bit)**

Interpretación de nombres en UNIX

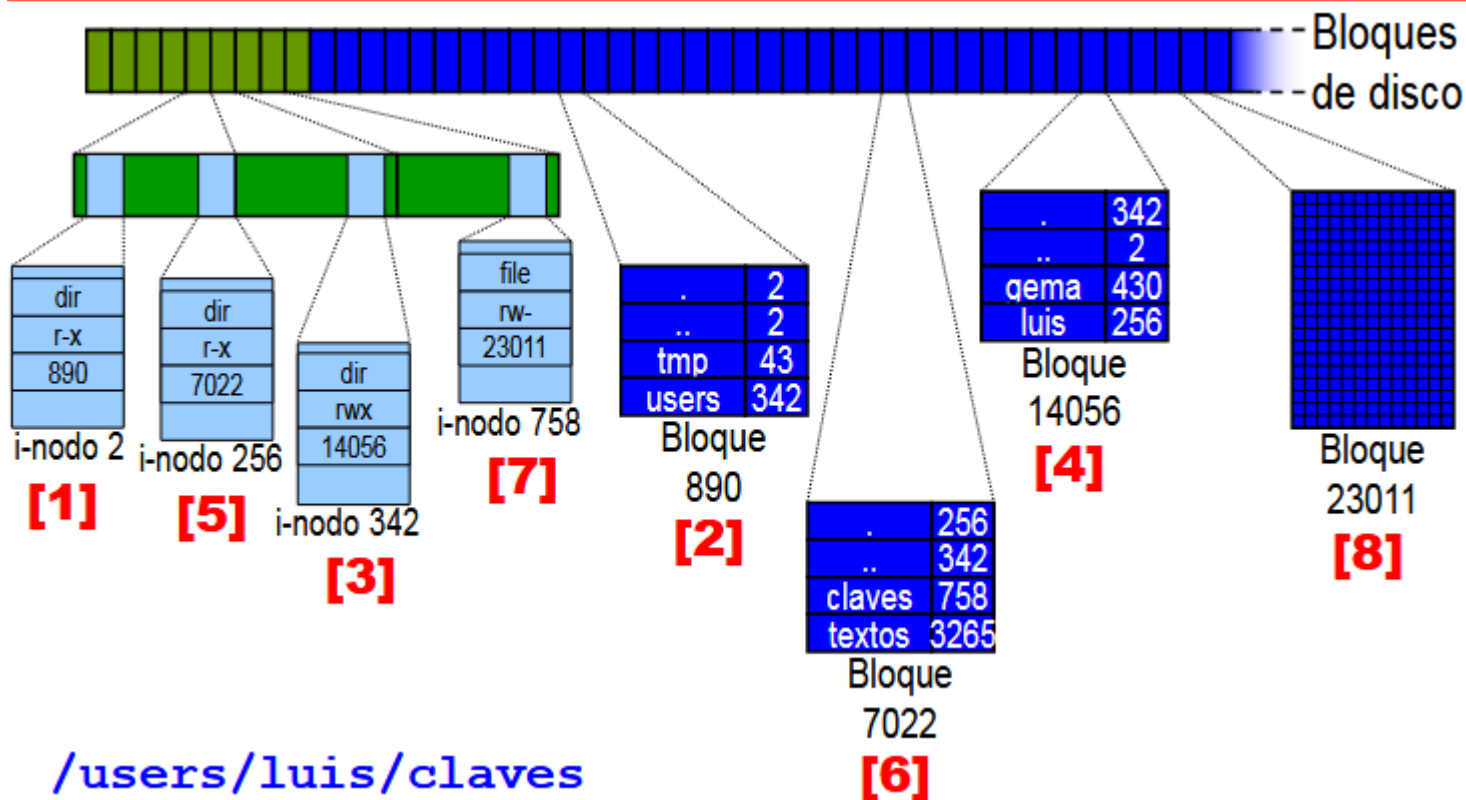
Interpretar /users/luis/claves

- [1] Traer a memoria entradas archivo con i-nodo 2
- [2] Se busca dentro users y se obtiene el i-nodo 342
- [3] Traer a memoria entradas archivo con i-nodo 342
- [4] Se busca dentro luis y se obtiene el i-nodo 256
- [5] Traer a memoria entradas archivo con i-nodo 256
- [6] Se busca dentro claves y se obtiene el i-nodo 758
- [7] Se lee el i-nodo 758 y ya se tienen dónde están los datos del archivo
- [8] Leer los bloques del fichero

¿Cuándo parar?

- No se tienen permisos
- Se ha encontrado el i-nodo del archivo
- No se ha encontrado y no hay más subdirectorios
- Estamos en un directorio y no contiene la siguiente componente del nombre (por ejemplo, miguel)

Gestión interna de archivo



Métodos de Acceso

Los SO ofrecen distintas formas de acceder a los bloques de información que hay dentro del archivo

- **Acceso secuencial**

- Metáfora: el fichero es una cinta secuencial. Solo se puede avanzar o retroceder
- Cuando se abre el archivo en un punto, las lecturas/escrituras sucesivas se van realizando a partir del último punto en el que se quedó
- Es el modelo básico de Unix y Windows (open -> read -> read -> ...)

- **Acceso directo**

- Metáfora: el fichero es un array de bloques de acceso aleatorio
- Cada operación de lectura/escritura indica la posición dentro del archivo sobre la que se quiere trabajar
- En Unix, se implementa con lseek()

Métodos de Acceso

- Acceso indexado ISAM

- Metáfora: el archivo es una tabla de una base de datos
- El archivo consiste en una tabla con registros, ordenados según una clave (ej. fichas de personas ordenadas por apellidos o CI)
- Cada operación de lectura se puede hacer buscando por clave, o por número de orden. Cada operación de escritura mantiene el archivo bien ordenado
- El SO asocia a cada archivo uno o varios *archivos índice* para mantener la ordenación de forma eficiente. Estos índices están ocultos al usuario
- Soportado en sistemas antiguos (ej. IBM OS/360, OpenVMS). Actualmente se implementa fuera del núcleo

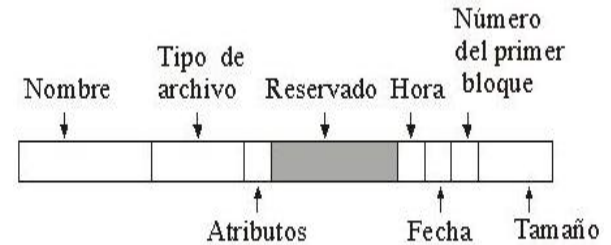
Directorios

Conceptos:

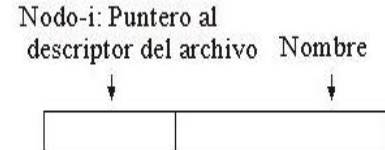
- Objeto que relaciona de forma unívoca un nombre de usuario de archivo con su descriptor interno
- Organizan y proporcionan información sobre la estructuración de los sistemas de archivo
- Una colección de nodos que contienen información sobre los archivos

Organización del directorio:

- Eficiencia: Localizar un archivo rápidamente
- Nombrado: Conveniente y sencillo para los usuarios:
 - Dos usuarios pueden tener el mismo nombre para archivos distintos
 - Los mismos archivos pueden tener nombres distintos
 - Nombres de longitud variable
- Agrupación: agrupación lógica de los archivos según sus propiedades
- Estructurado: operaciones claramente definidas y ocultación
- Sencillez: la entrada de directorio debe ser simple



Directorio de MS-DOS

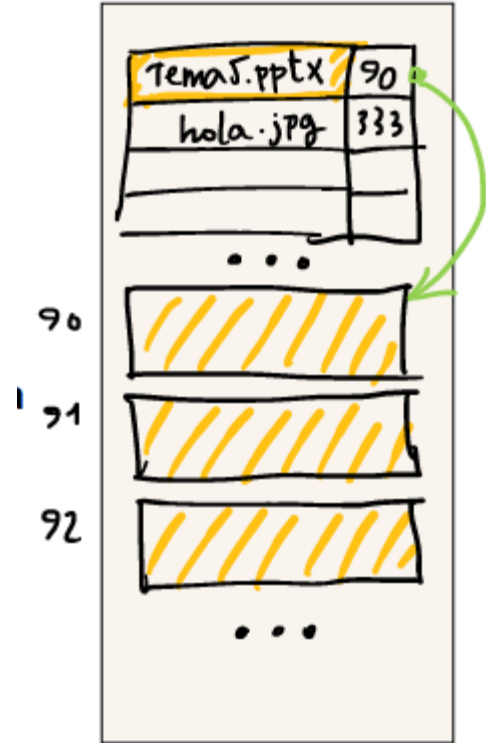


Directorio de UNIX

Directorios

Dos casos:

1. Un usuario quiere trabajar con el documento “tema5.pptx”. ¿Cómo sabe el SO que ese archivo existe? ¿Dónde está ubicado en el disco?
2. Un usuario quiere saber qué archivos contiene un disco (obtener un inventario de los nombres de esos archivos)
 - El SO debe manejar una estructura con el catálogo de los archivos -> **directorio o tabla de contenidos**
 - El directorio debe estar almacenado en el disco (por eficiencia, se puede tener una copia en RAM mientras se trabaja con él)
 - Cada entrada en el directorio contiene los atributos de un archivo y su localización en el disco



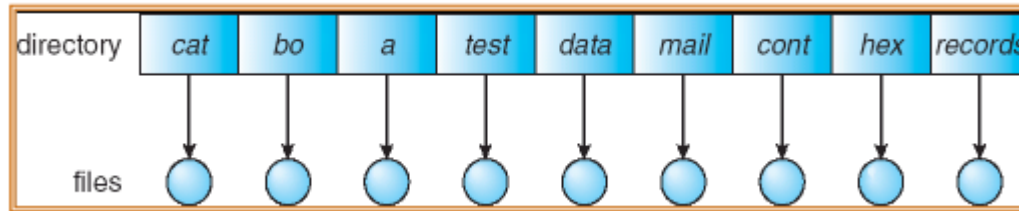
Operaciones sobre directorios

- Buscar un archivo por nombre
- Listar el contenido del directorio
- Crear un archivo vacío
- Eliminar un archivo
- Renombrar un archivo
- Copiar/mover un archivo de un directorio a otro
- Guardar una versión de un archivo

Modelos estructurales

Directorio de nivel único

- Todos los archivos se guardan en un directorio principal (*volume table of contents*)



Limitaciones

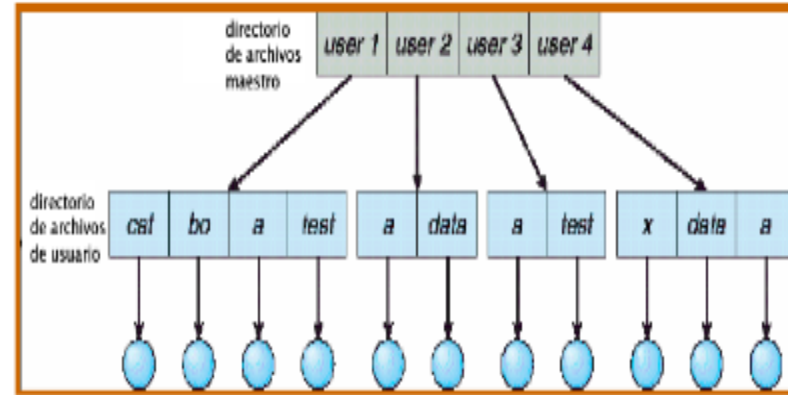
- No se adapta bien a sistemas multiusuario. Ej. conflictos de nombres, permisos de acceso, etc.
- Si hay muchos archivos, crece el tiempo de búsqueda en el volumen

Modelos estructurales

Directorio de dos niveles

- El directorio principal no contiene archivos, solo subdirectorios. Estos a su vez contienen los archivos
- Los directorios se pueden usar para separar cuentas de usuario, aplicaciones, archivos del sistema, etc.
- Aparece el concepto de *ruta (path)*, ej.
user1:data

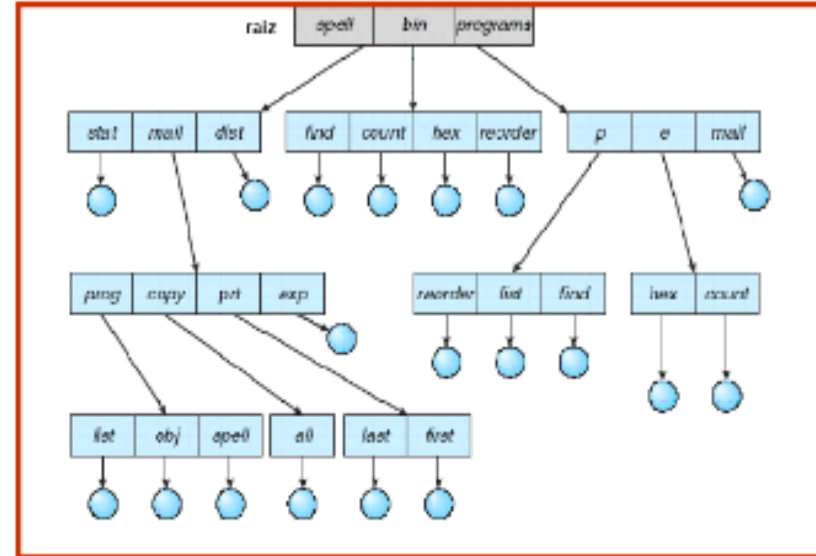
Ventajas: mejor organización de los archivos;
se pueden definir permisos por cada directorio;
se evitan conflictos de nombres...



Modelos estructurales

Estructura de árbol

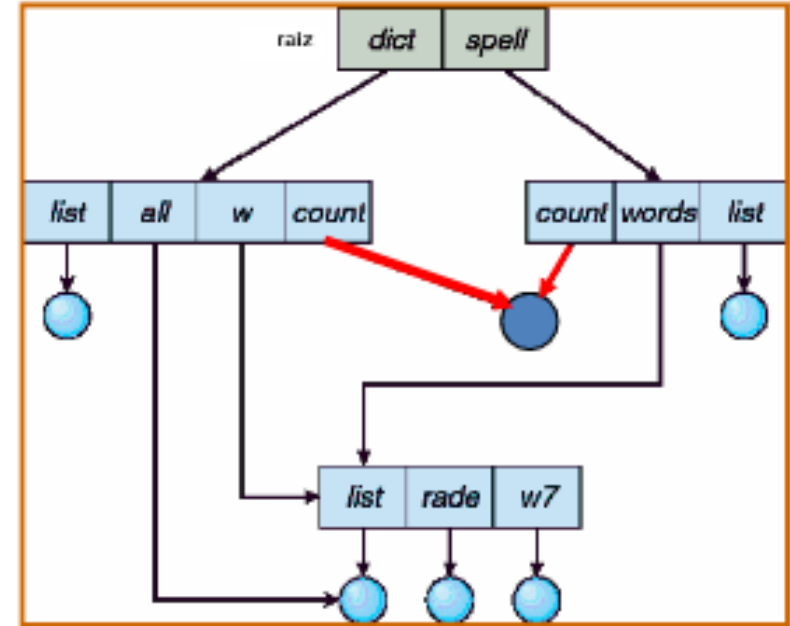
- Generalización recursiva del concepto de «subdirectorio»
- Un directorio puede contener archivos y otros directorios
- «ruta» (pathname) -> localiza un objeto dentro del árbol. – Ej. /programs/p/find
- Conceptos útiles:
 - directorio de trabajo
 - rutas absolutas o relativas
- ¿Cómo interpretamos la eliminación de un directorio? ¿Borramos todo su subárbol?



Modelos estructurales

Grafos Acíclicos: enlaces

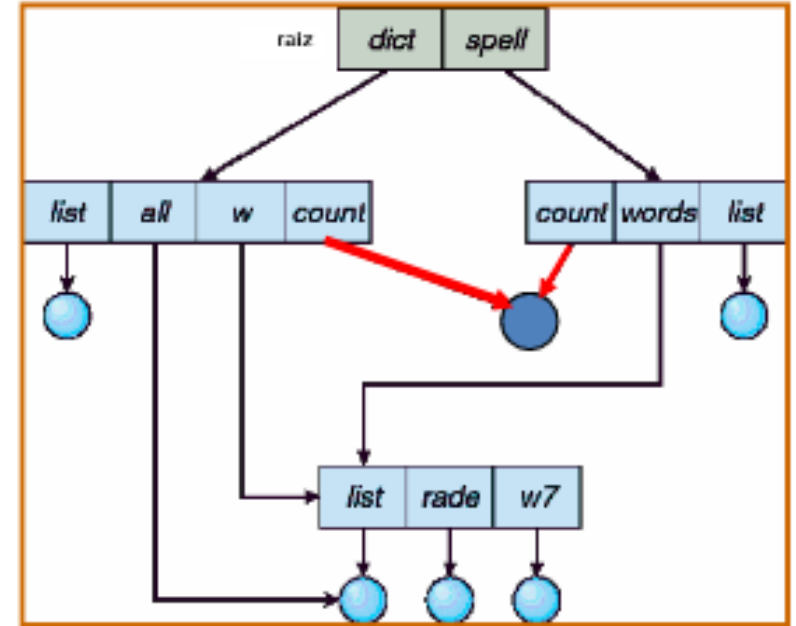
- Idea: permitir que un mismo objeto esté enlazado desde varias rutas.
 - `/dict/count = /spell/count`
- Resultado: el directorio tiene forma de grafo encíclico
- Implementación (Unix):
 - Enlaces «duros»
 - Enlaces simbólicos:
 - `ln -s /a/b/fich/c/d/alias`



Modelos estructurales

Grafos Acíclicos: enlaces

- ¿Qué hacer cuando se pide eliminar un objeto compartido?
- ¿Qué pasa si se hace un recorrido por el grafo? ¿Cómo se sabe si ya se ha pasado por un objeto duplicado?



Modelos estructurales

Enlace o link: Un archivo con varios nombres -> control de enlaces

- Un único archivo con contador de enlaces en el descriptor (e. físicos)
- Archivos nuevos con el nombre destino dentro (e. simbólicos)

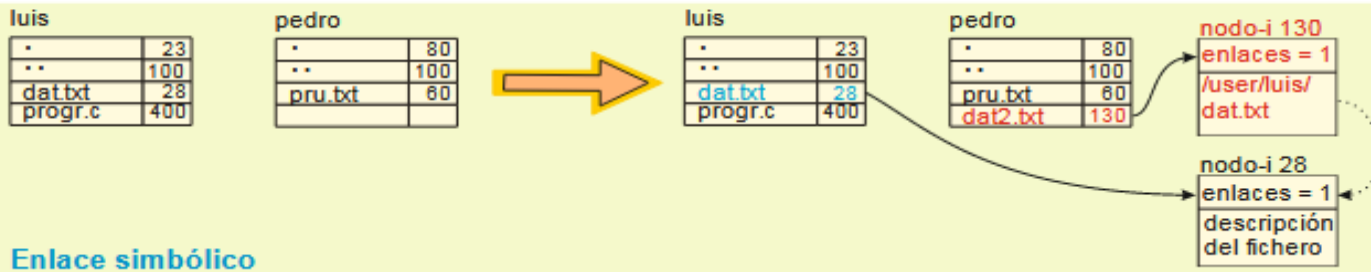
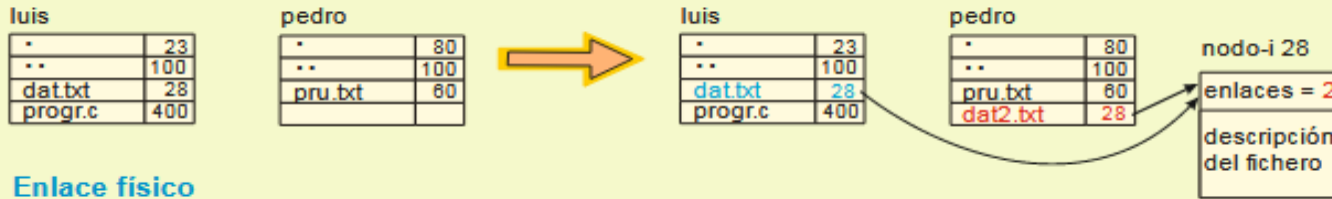
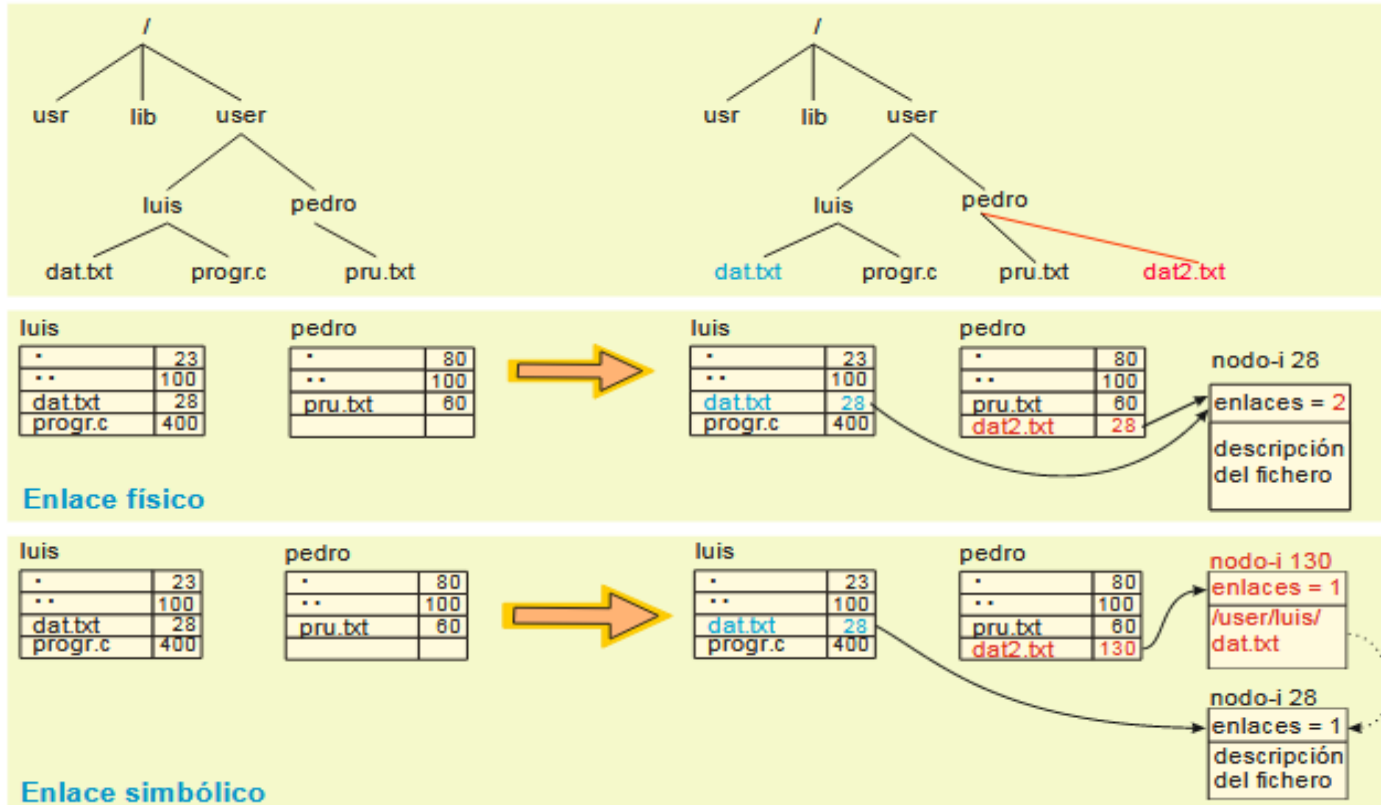
Borrado de enlaces:

- Decrementar el contador; si 0 borrar archivo
- Recorrer los enlaces y borrar todos
- Borrar únicamente el enlace y dejar los demás

Problema grave: existencia de bucles en el árbol

- Permitir solo enlaces a archivos no a subdirectorios
- Algoritmo de búsqueda de bucle cuando se hace un enlace
- Limitación en LINUX solo enlaces físicos dentro del mismo sistema de archivos

2) Enlaces físicos y simbólicos



El Sistema de Archivos SA

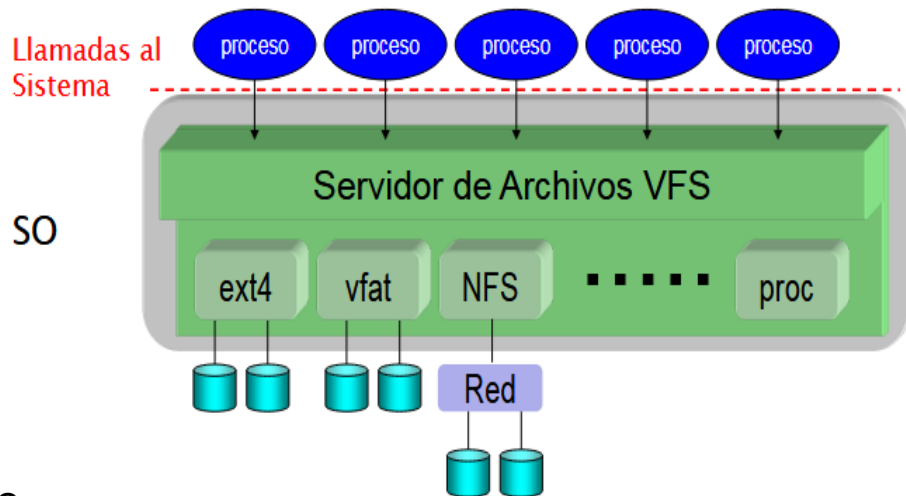
Los archivos “regulares” tienen que almacenar sus datos y metadatos en soporte físico

La visión de los archivos “en uso” requiere:

- Esos datos y metadatos
- Información de ejecución
- Toda la lógica de gestión

Se puede diferenciar entre:

- Interfaz del SrvArch hacia al usuario
- Interacción con los dispositivos



> VFS: Virtual File System

Sistema de Archivos Virtual (VFS: Virtual File System):

- Interfaz de llamadas de entrada/salida del sistema (hacia el exterior)
- Módulo de organización de archivos (internamente)

Servicios

- Gestión de nombres (e.g.: link)
- Algunos servicios de seguridad (e.g: chmod)
- Integración dentro del SrvArch de distintos tipos de sistemas de archivos (registro)
- Servicios genéricos de archivos y directorios (e.g: read, creat, mkdir)

Nodo virtual (v-nodo)

- Estructura de información
- Común a todos los sistemas de archivos subyacentes
- Enlaza con un descriptor de archivo de cada tipo particular

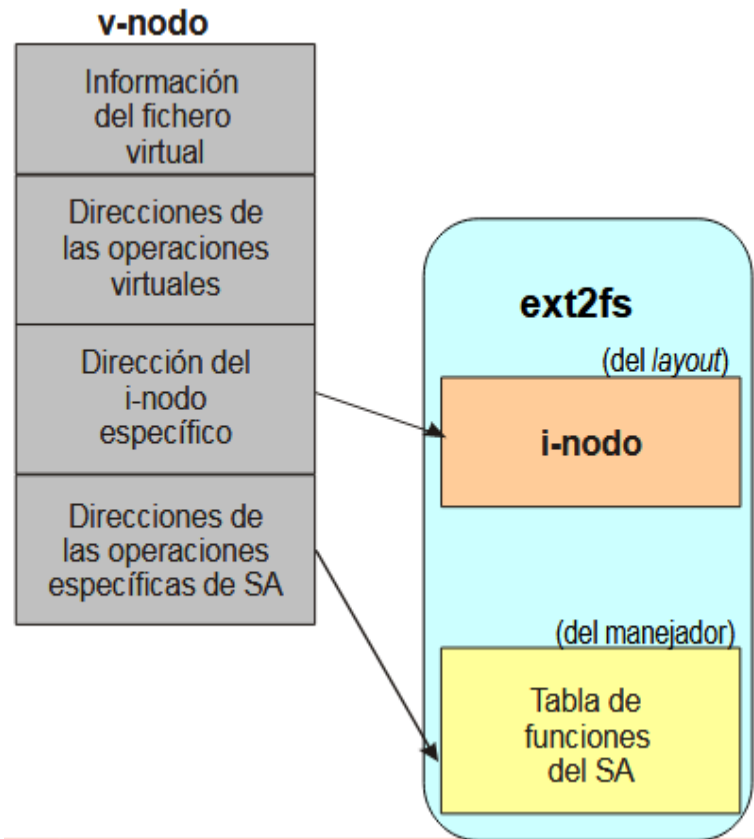
2) VFS: Virtual File System

v-nodo: Estructura en memoria del VFS que gestiona las estructuras de fichero para cualquier sistema de archivos

Superclase de todos los tipos posibles de i-nodos

Cuando se hace referencia a un archivo se usa una estructura de este tipo, compuesta por:

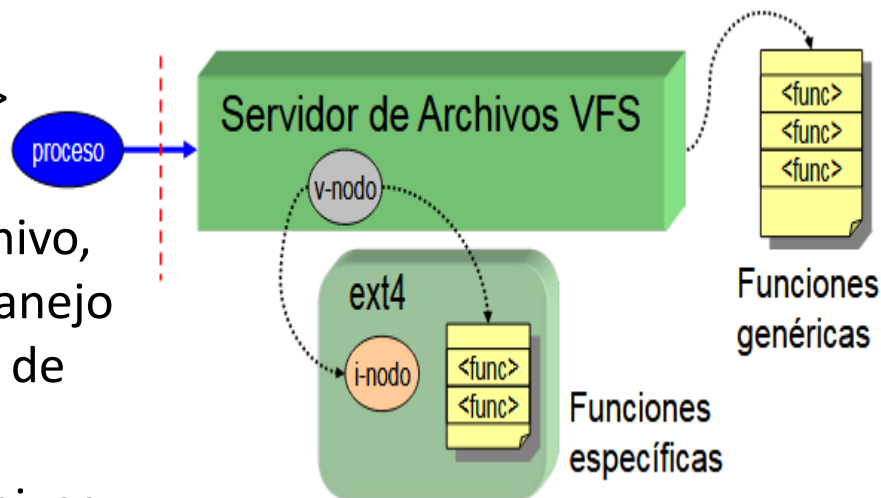
- El i-nodo del soporte físico o equivalente
- Variables de gestión (semáforos, contadores, colas)
- Referencia al dispositivo de bloque que lo soporta



VFS: Virtual File System

Proporciona el modelo de archivo y sus servicios:

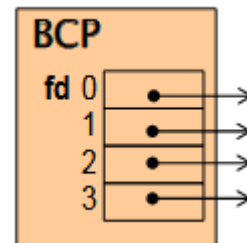
- **Algoritmos:** Imagen lógica del fichero -> imagen física
- **Gestión:** Espacio de los sistemas de archivo, asignación de bloques a archivos y el manejo de los descriptores de archivos (i-nodos de UNIX o registros de Windows NT)
- **Soporte:** A cada tipo de Sistema de Archivos (ext2fs, ntfs, hpfs, ...)



VFS: Virtual File System

Tabla de archivos abiertos (tdaa) por proceso, dentro del PCB, con sus descriptores temporales y el valor del apuntador de posición del archivo para ese proceso.

- El tamaño de esta tabla define el máximo número de archivos que cada proceso puede tener abierto de forma simultánea
- El descriptor de archivo fd indica el lugar de tabla. La tdaa se rellena de forma ordenada, de forma que siempre se ocupa la primera posición libre de la tabla
- En los sistemas UNIX cada proceso tiene tres descriptores de archivos abiertos por defecto: entrada estándar, fd = 0, salida estándar, fd = 1, error estándar, fd = 2



VFS: Virtual File System

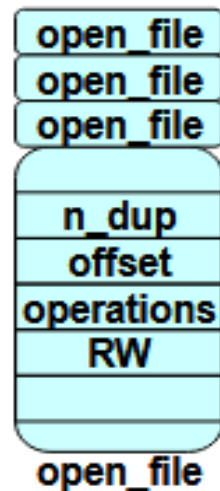
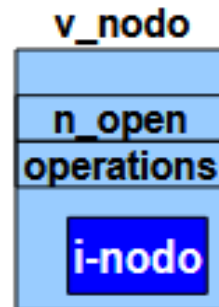
Tabla de v-nodos: Almacena en memoria información del i-nodo (i) que existente en el disco y (ii) la que se usa dinámicamente (sólo tiene sentido cuando el archivo está en uso [e.g. cerrojos]).

Problema: si sólo hay un campo de apuntador, cada operación de un proceso afectaría a todos los demás

Posible solución: incluir la información relativa al archivo dentro del bloque de descripción del proceso (PCB)

Tabla intermedia v-nodo-posición: entre la tabla de archivos del PCB y la tabla de v-nodos. Dicha tabla incluye, entre otras cosas:

- La entrada del v-nodo del archivo abierto en la tabla de v-nodos.
- El apuntador de posición correspondiente al proceso, o procesos, que usan el archivo durante esa sesión.
- El modo de apertura del archivo
- File operations



Sistemas de archivos y particiones

El sistema de archivos permite organizar la información dentro de los dispositivos de almacenamiento secundario en un formato inteligible para el sistema operativo

- Previamente a la instalación del sistema de archivos es necesario dividir físicamente, o lógicamente, los discos en particiones o volúmenes [Pinkert 1989]
- Una partición es una porción de un disco a la que se la dota de una identidad propia y que puede ser manipulada por el sistema operativo como una entidad lógica independiente
- Una vez creadas las particiones, el sistema operativo debe crear las estructuras de los sistemas de archivos dentro de esas particiones. Para ello se proporcionan mandatos como `format` o `mkfs` al usuario
 - `#mkswap -c /dev/hda2 20800`
 - `#mkfs -c /dev/hda3 -b 8196 123100`

El Master Boot Record MBR

El MBR es el primer sector del disco y el mismo para casi todos los sistemas operativos

Offset	Descripción	Tamaño
000h	Executable Code (Boots Computer)	446 Bytes
1Beh	1st Partition Entry	16 Bytes
1Ceh	2nd Partition Entry	16 Bytes
1Deh	3rd Partition Entry	16 Bytes
1Eeh	4th Partition Entry	16 Bytes
1Feh	Boot Record Signature (55h AAh)	2 Bytes

Offset	Descripción	Tamaño
00h	Current State of Partition (00h=Inactive, 80h=Active)	1 Byte
01h	Beginning of Partition - Head	1 Byte
02h	Beginning of Partition - Cylinder/Sector	1 Word
04h	Type of Partition	1 Byte
05h	End of Partition - Head	1 Byte
06h	End of Partition - Cylinder/Sector	1 Word
08h	Number of Sectors Between the MBR and the First Sector in the Partition	1 Double Word
0Ch	Number of Sectors in the Partition	1 Double Word

Tipos de partición: Fat-12, Fat-16, Fat-32, XENIX, NTFS, AIX, Linux, MINIX, BSD ...

Ejemplos de SA

Descripción del layout de un SA:

- El sector de arranque en MS-DOS
- El superbloque en UNIX

Relación SA-dispositivo:

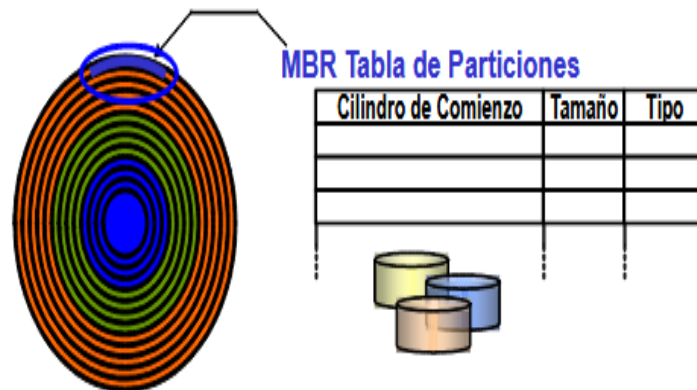
- **Típico**: 1 dispositivo N sistemas de archivos (particiones)
- **Grandes archivos**: N dispositivos 1 sistema de archivos

Típicamente cada dispositivo se divide en una o más particiones

(en cada partición sistema de archivos)

La tabla de particiones indica el principio, el tamaño y el tipo de cada partición

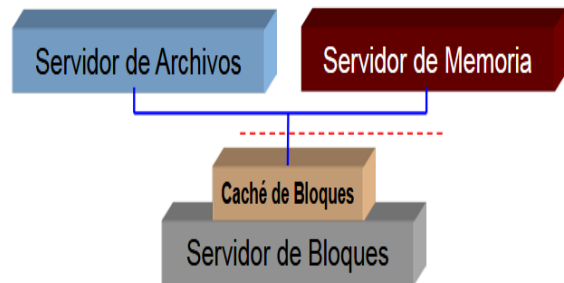
FAT12 y 16	Boot	FAT	FAT	Dir. Raíz	Datos y directorios
FAT32	Boot	FS info	FAT	FAT	Datos y directorios
UNIX	Boot	Super Blq.	Bit maps	i-nodos	Datos y directorios
NTFS	Boot	Super Blq.	Bit maps	Desc. físicos	Datos y directorios



Servidor de bloques

Se encarga de emitir los mandatos genéricos para leer y escribir bloques a los manejadores de dispositivo

- La E/S de bloques de archivo, y sus posibles optimizaciones, se llevan a cabo en este nivel del SA
- Las operaciones se traducen a llamadas de los manejadores de cada tipo de dispositivo específico y se pasan al nivel inferior del sistema de archivos
- Esta capa oculta los distintos tipos de dispositivos, usando nombres lógicos para los mismos. Por ejemplo, /dev/hda3 será un dispositivo de tipo hard disk (hd), cuyo nombre principal es a y en el cual se trabaja sobre su partición 3
- Los mecanismos de optimización de la E/S, como la cache de bloques, se incluye en este nivel



Un servidor de bloques proporciona servicio a:

- **Servidor de Archivos:** Bloques de datos de los archivos
- **Servidor de Memoria:**
 - Proyección de ficheros en memoria (carga de ejecutables)
 - Gestión de swap/fallos de página

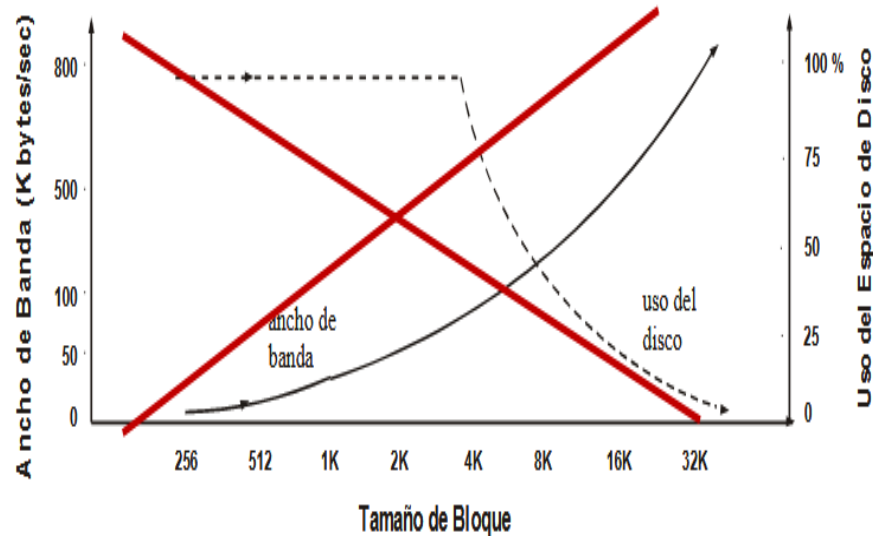
Tamaño del bloque

Bloque: agrupación lógica de sectores de disco y es la unidad de transferencia mínima del SA

- Optimizar la eficiencia de la E/S de los dispositivos secundarios
- Todos los SO proporcionan un tamaño de bloque por defecto
- Los usuarios pueden definir el tamaño de bloque a usar dentro de un SA con *mkfs*.

Agrupación: conjunto de bloques que se gestionan como una unidad lógica de almacenamiento.

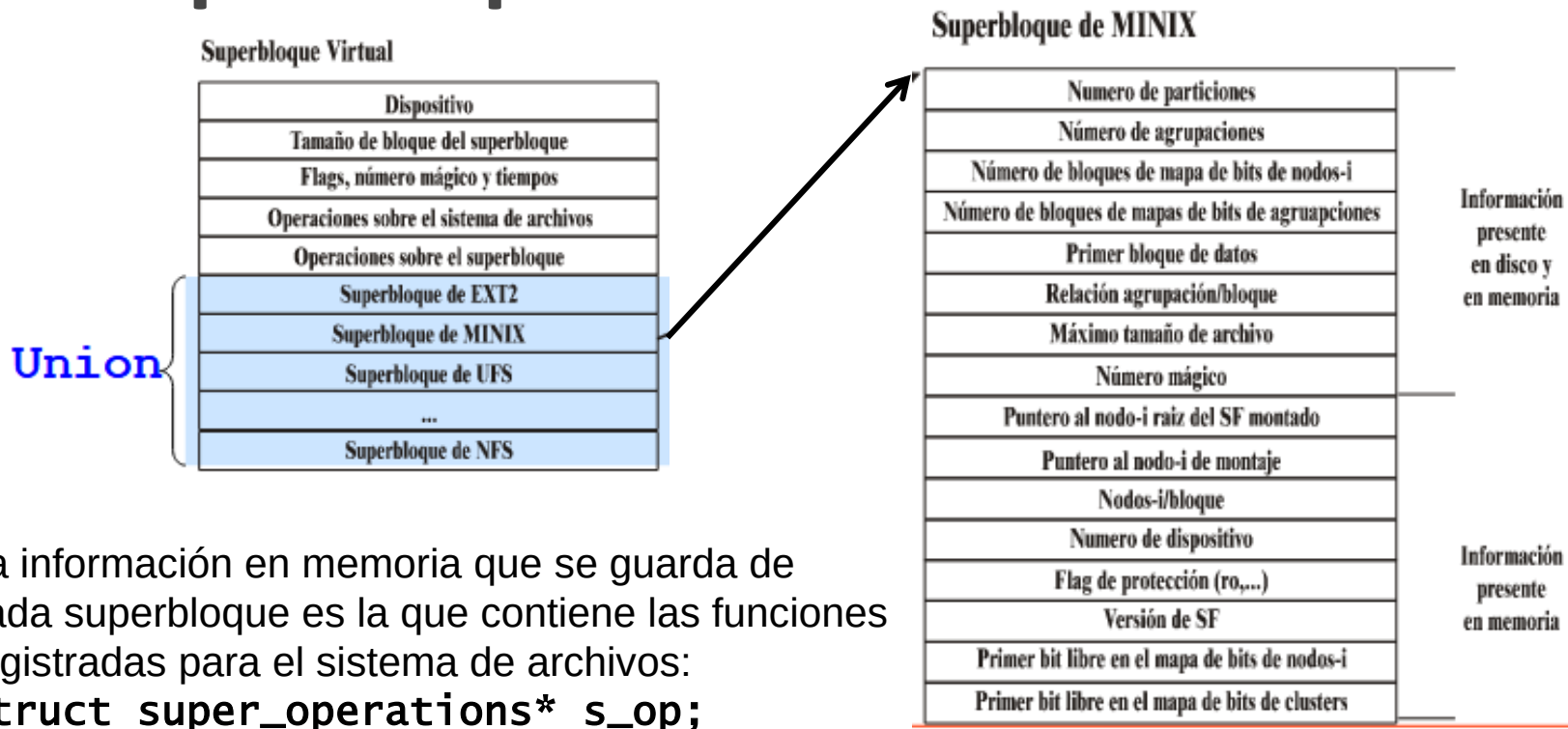
- El problema que introducen las agrupaciones, y los bloques grandes, es la existencia de fragmentación interna.
- Unidad de asignación. Puede ser 1 agrupación=1 bloque



Al aumentar el tamaño del bloque:

- Aumenta el ancho de banda efectivo del disco
- Disminuye el uso efectivo del espacio del disco

El Superbloque



Implementación del SA

Métodos de asignación de espacio

Cada archivo debe tener asignado un conjunto de bloques de datos

¿Cómo elegimos cuáles son esos bloques?

Varias técnicas básicas:

- Asignación contigua
- Asignación enlazada
- Asignación indexada

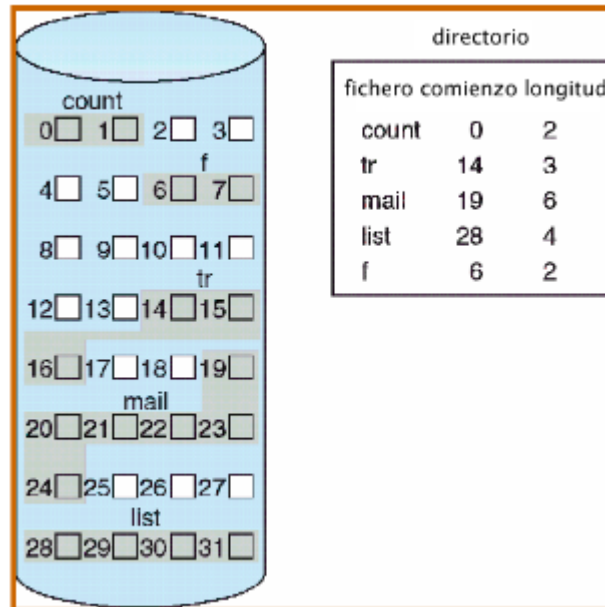
Casos particulares:

- FAT (variante de la enlazada)
- Contigua con extensiones (extents)

Métodos de Asignación de Espacio

Asignación Contigua

- Cada archivo ocupa un conjunto de bloques contiguos en el disco
- Entrada de directorio para cada archivo:
 - Dirección del bloque inicial
 - Longitud del archivo (de ahí se deduce el número de bloques)



Métodos de Asignación de Espacio

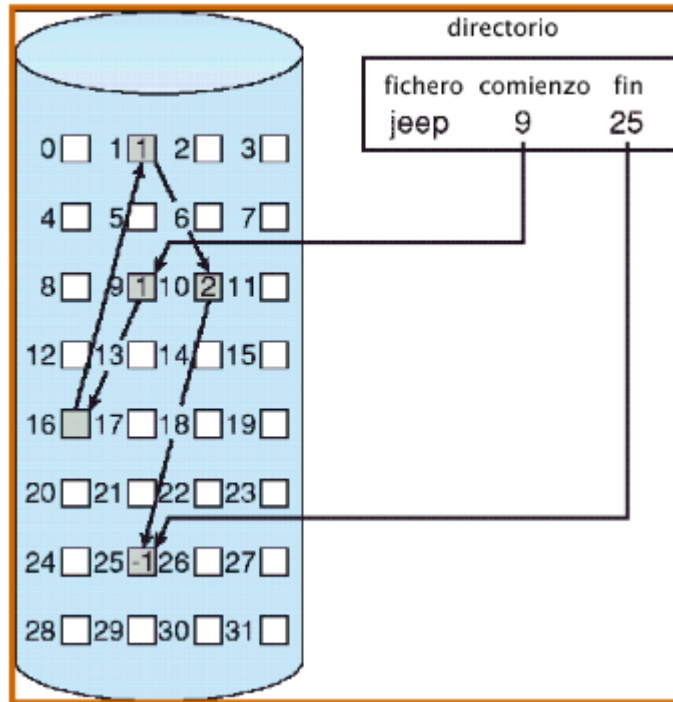
Asignación Contigua Pros y Contras

- Permite implementar muy eficientemente cualquier acceso al archivo (secuencial o directo)
- Como los bloques están contiguos, los recorridos por el archivo no implican mucho movimiento de los cabezales (en discos magnéticos)
- **Problema 1: *fragmentación*.** ¿Qué pasa si no encuentro un número de bloques contiguos suficientes para el archivo?
- ¡El mismo problema que teníamos en la gestión de la memoria principal!
- **Problema 2:** ¿cómo abordamos el crecimiento progresivo de un archivo? ¿Reservamos el máximo de bloques desde el principio? ¿O vamos reservando espacio a medida que el archivo crece?

Métodos de Asignación de Espacio

Asignación Enlazada

- Los bloques del archivo no tienen por qué ser contiguos.
- Se forma una lista enlazada de bloques.
- La entrada en directorio contiene:
 - Dirección del bloque inicial
 - Longitud del archivo
 - A veces, dirección del bloque final (para que sea fácil hacer crecer el archivo)



Métodos de Asignación de Espacio

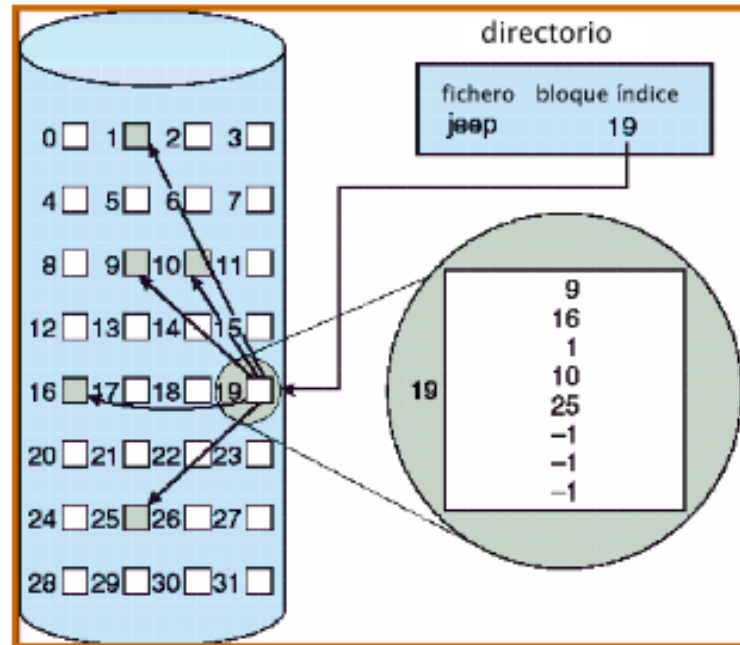
Asignación Enlazada Pros y Contras

- Respecto a la asignación contigua, soluciona radicalmente los problemas de la fragmentación y de la reserva previa de espacio
- **Inconvenientes:**
 - El acceso directo es más costoso: hay que recorrer enlaces hasta llegar al bloque deseado
 - Se consume espacio para los enlaces de la lista
 - **Fragilidad:** si se daña un bloque a mitad del archivo, se pierde el acceso al resto del archivo
- Algunas variantes para aumentar rendimiento y robustez:
 - Enlaces dobles (hacia delante y hacia atrás)
 - En cada bloque, almacenar un «retroenlace»: apuntador a la entrada en directorio

Métodos de Asignación de Espacio

Asignación Indexada

- **La idea:** reunir todos los enlaces del archivo en un bloque de índices separado
 - La entrada K del bloque de índices contiene la dirección del bloque K del archivo
- La entrada en directorio tiene:
 - Dirección del bloque de índices
 - Longitud del archivo (se deduce cuántas entradas útiles tiene el bloque de índices)



Métodos de Asignación de Espacio

Asignación Indexada Pros y Contras

- Igual que la asignación enlazada, no tiene problemas de fragmentación externa
- El acceso directo se realiza en un tiempo constante (solo necesita acceder al bloque de índices y luego al bloque de datos del archivo)
- Requiere un bloque de datos adicional (el bloque de índices). Esto impacta más en archivos de pequeño tamaño
- Con respecto a la asignación contigua, requiere un acceso adicional a disco (al bloque de índices) -> pero el bloque de índices se puede guardar en caché
- **Fragilidad:** si se daña el bloque de índices, se pierde todo el contenido del archivo

Métodos de Asignación de Espacio

Asignación Indexada: Tamaño del bloque de índices

¡OJO! El tamaño del bloque de datos limita el número de índices, y con ello limita el tamaño máximo que puede tener un archivo

Ejemplo:

Tenemos esta configuración:

- Tamaño de bloque de datos = 1KiB = 1024 bytes
- Tamaño de un enlace = 16 bits = 2 bytes

Con esta configuración, ¿cuánto puede llegar a medir un archivo?

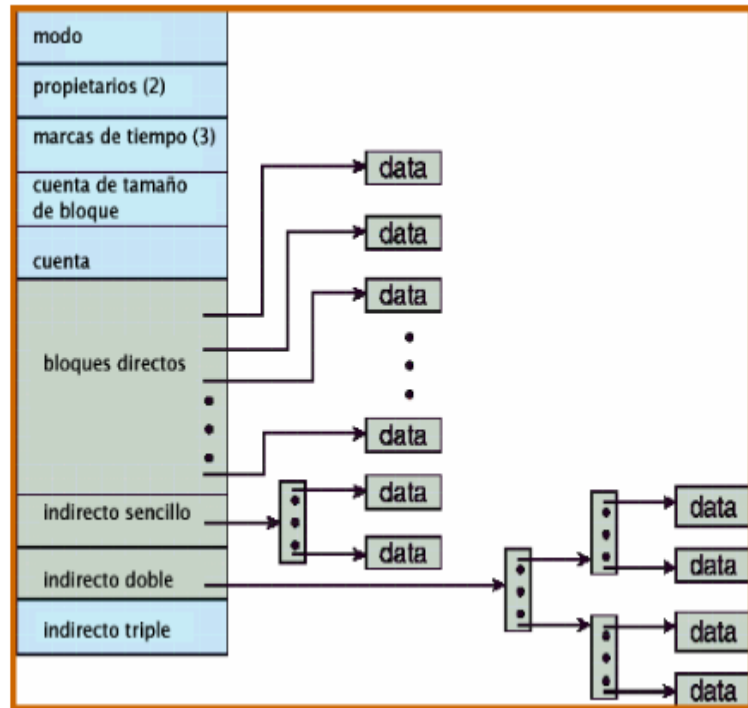
- En un bloque de índices caben $1024/2 = 512$ entradas
- Por tanto, un archivo puede tener hasta 512 bloques
- Como cada bloque mide 1KiB, el tamaño máximo de un archivo = $512 \cdot 1\text{KiB} = 512\text{KiB}$

Métodos de Asignación de Espacio

Asignación Indexada: Tamaño del bloque de índices

¿Qué podemos hacer para resolver la limitación de tamaño del bloque de índices?

- Permitir que los índices ocupen varios bloques contiguos -> vuelve la fragmentación
- Lista enlazada de índices -> ineficiente
- Estructura multinivel (bloque de índices que apunta a un segundo nivel de bloques de índices) -> esta fue la solución de UNIX



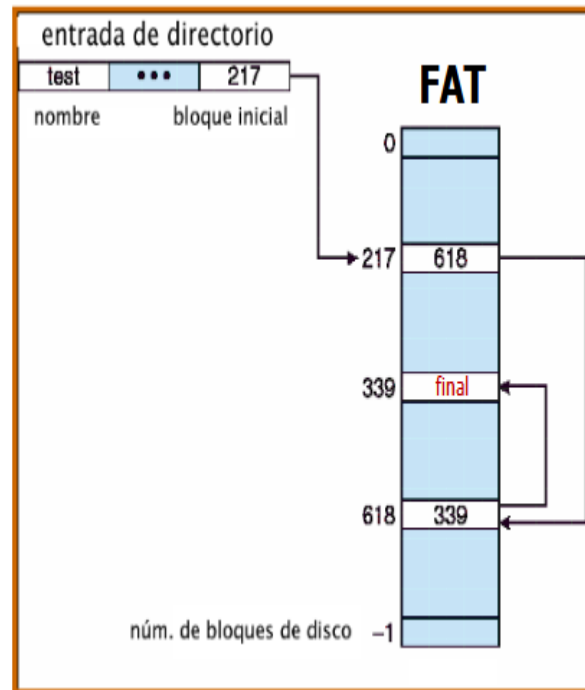
Métodos de Asignación de Espacio

Asignación Enlazada con FAT (*File Allocation Table*)

- Variación del método de asignación enlazada
- Creada para MS-DOS -> continúa usándose en las memorias flash (VFAT, ExFAT)

La idea: reunir todos los enlaces en una única estructura de datos compacta dentro del disco

- Se reserva una zona al principio del disco al principio para guardar la tabla
- Cada entrada de la tabla corresponde a un bloque. Su contenido dice dónde está el siguiente bloque
- Los enlaces del fichero “test” están en la FAT:
 - 217 -> 618 -> 339
 - El estándar tiene un código especial para marcar “fin de fichero”



Métodos de Asignación de Espacio

Asignación Enlazada con FAT (*File Allocation Table*)

- Respecto a la asignación enlazada, mejora el tiempo de acceso general (facilita guardar varios enlaces en una caché)
- En dispositivos pequeños, la FAT completa se puede traer a la RAM. Esta caché acelera los accesos a los archivos
- Para recorrer un archivo, solo hacen falta los bloques de la FAT en los que está la información de los enlaces del archivo. Si esos bloques están bastante cercanos entre sí, no hará falta leer muchos bloques
- **Fragilidad:** si un bloque de la FAT se daña, se pierden numerosos enlaces de múltiples archivos -> **Solución:** el disco tiene varias copias de la FAT

Métodos de Asignación de Espacio

Asignación Contigua con Extents

- Los SF más modernos (NTFS, XFS...) utilizan un híbrido entre la asignación contigua y las no contigua
- Si se puede, el archivo utiliza un conjunto contiguo de bloques de datos (igual que la asignación contigua)
- Si no basta con un espacio contiguo, se busca otra extensión de bloques contiguos
- En la entrada en directorio se guarda la información de todas las extensiones. Por cada extensión: bloque inicial, nº bloques.
- El algoritmo para acceder a un punto aleatorio del archivo es más complejo, pero en general mantiene un nivel bajo de fragmentación y se consume muy poco espacio en estructuras de control.
- Empíricamente se observa que la mayoría de los archivos solo necesitan una extensión, así que en la práctica tiene casi el mismo rendimiento que la asignación contigua

1) Gestión del Espacio Libre

Mapas de bits, o vectores de bits: un bit por recurso existente (descriptor de archivo, bloque o agrupación). Si el recurso está libre, el valor del bit asociado al mismo es 1, si está ocupado es 0

Ej.: Sea un disco en el que los bloques 2, 3, 4, 8, 9 y 10 están ocupados y el resto libres, y en el que los descriptores de archivo 2, 3 y 4 están ocupados. Sus mapas de bits serán:

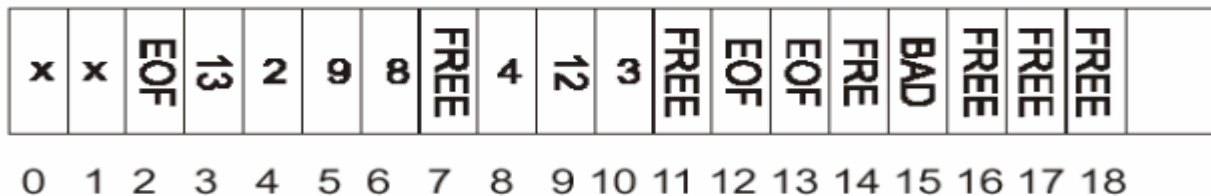
- MB de bloques: 1100011100011.... M bloques en disco
- MB de descriptores: 1100011... i-nodos

Fácil de implementar y sencillo de usar. Eficiente si el dispositivo no está muy lleno o muy fragmentado

1) Gestión del Espacio Libre

Listas de recursos libres: mantener enlazados en una lista todos los recursos disponibles (bloques o descriptores de archivos) manteniendo un apuntador al primer elemento de la lista.

- Este método no es eficiente, excepto para dispositivos muy llenos y fragmentados
- Uso de agrupaciones.



Bloques libres → 7 → 11 → 14 → 16 → 17 → 18 (A)

Bloques libres → 7,1 → 11,1 → 14,1 → 16,3 (B)

Mejoras en el rendimiento del SA

Basados en el uso de almacenamiento intermedio de datos de entrada/salida en memoria principal. Estos mecanismos son de dos tipos:

Discos RAM: sus datos están almacenados sólo en memoria principal.

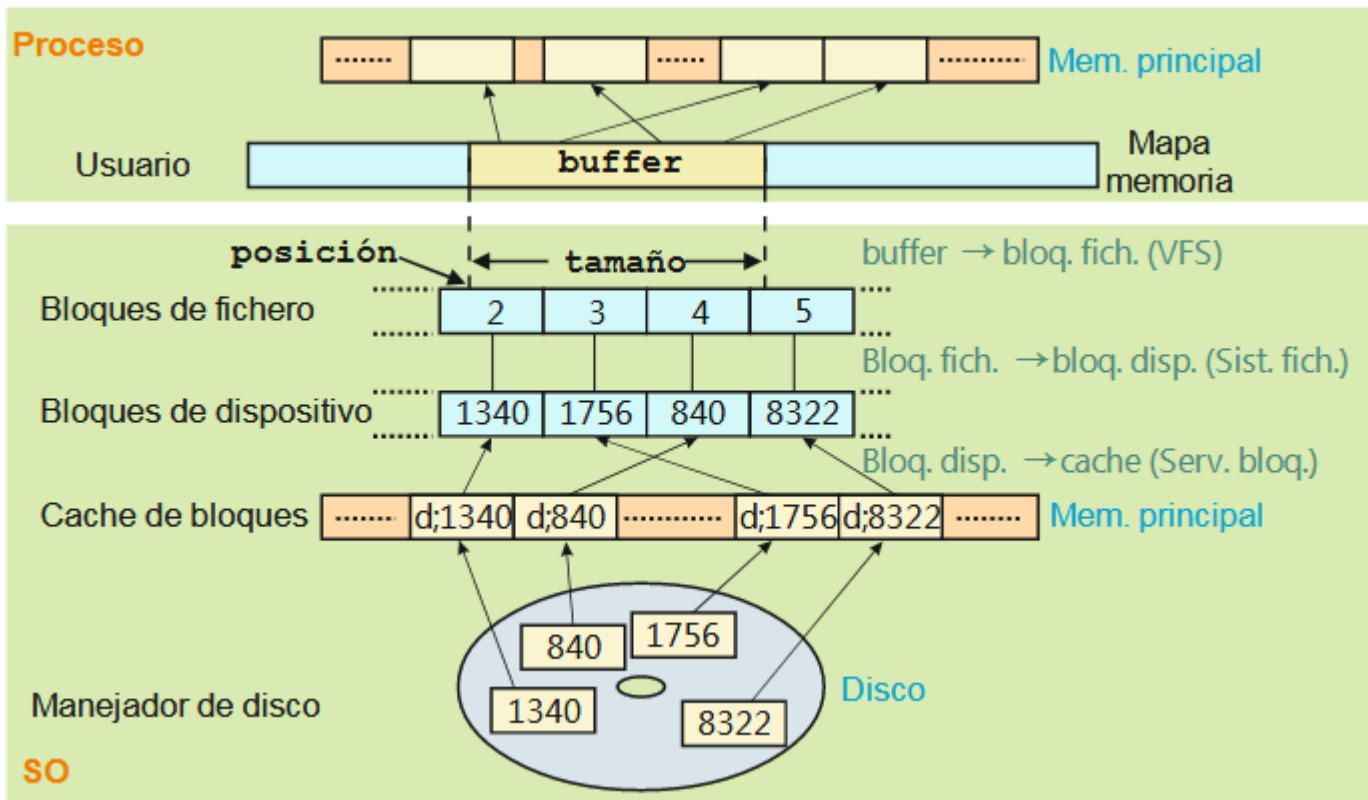
- Aceptan todas las operaciones de cualquier otro sistema de archivos y son gestionados por el usuario.
- Pseudodispositivos para almacenamiento temporal o para operaciones auxiliares del sistema operativo. Su contenido es volátil.

⇒ **Cache de datos:** instaladas en secciones de memoria principal controladas por el sistema operativo, donde se almacenan datos para optimizar accesos posteriores

- Se basan en la existencia de proximidad espacial y temporal en las referencias a los datos de entrada/salida.
- Dos caches importantes dentro del servidor de archivos: cache de nombres y cache de bloques

Implementación del SA

Flujo de datos
en el SA



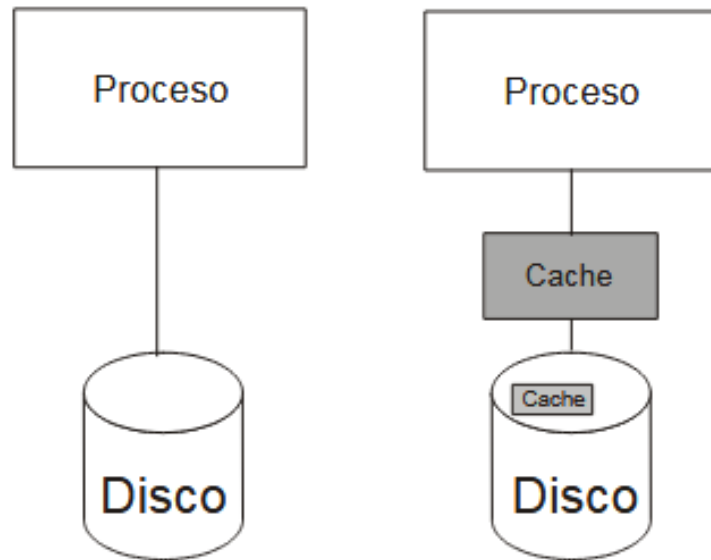
Mejoras en el rendimiento del SA

2) Caché de Bloques

Estructura de datos en memoria con los bloques más frecuentemente utilizados

- Lecturas adelantadas (acceso secuencial)
- Limpieza de la cache (sync)

Principal problema: fiabilidad del sistema de archivos



Mejoras en el rendimiento del SA

Caché de Bloques – Políticas de reemplazo

1) Algoritmo: Comprobar si el bloque a leer está en la cache.

- En caso de que no esté, se lee del dispositivo y se copia a la cache
- Si la cache está llena, es necesario hacer hueco para el nuevo bloque reemplazando uno de los existentes: políticas de reemplazo.
- Si el bloque ha sido escrito (sucio): política de escritura

2) Políticas de reemplazo: FIFO (First in First Out), segunda oportunidad, MRU (Most Recently Used), LRU (Least Recently Used), etc.

- La política de reemplazo más frecuentemente usada es la LRU. Esta política reemplaza el bloque que lleva más tiempo sin ser usado, asumiendo que no será referenciado próximamente.
- Los bloques más usados tienden a estar siempre en la cache y, por tanto, no van al disco. La utilización estricta de esta política puede crear problemas de fiabilidad en el sistema de archivos si el computador falla.
- La mayoría de los servidores de archivos distinguen entre bloques especiales y bloques de datos

Mejoras en el rendimiento del SA

) Caché de Bloques – Políticas de escritura

Escritura inmediata (write-through): se escribe cada vez que se modifica el bloque

- No hay problema de fiabilidad, pero se reduce el rendimiento del sistema

Escritura diferida (write-back): sólo se escriben los datos a disco cuando se eligen para su reemplazo por falta de espacio en la cache

- Optimiza el rendimiento, pero genera los problemas de fiabilidad anteriormente descritos

Escritura retrasada (delayed-write): consiste en escribir a disco los bloques de datos modificados en la cache de forma periódica cada cierto tiempo (30 segundos en UNIX).

- Compromiso entre rendimiento y fiabilidad
- Reduce la extensión de los posibles daños por pérdida de datos
- Los bloques especiales se escriben inmediatamente al disco
- No se puede quitar un disco del sistema sin antes volcar los datos de la cache

Escritura al cierre (write-on-close): cuando se cierra un archivo, se vuelcan al disco los bloques del mismo que tienen datos actualizados

Mejoras en el rendimiento del SA

2) Habitualmente en el acceso a un archivo hay que acceder a dos partes del disco:

- Entrada del descriptor (i-nodo)
- Bloques de datos

Por cuestiones de eficiencia se considera más apropiado hacer que todos estos accesos se realicen sobre zonas próximas dentro del disco

Solución: Intercalar bloques de i-nodos con sus bloques de datos relacionados.

Implementaciones: Extended 2 Filesystem (ext2), ext3 y ext4

Mejoras en el rendimiento del SA

En los sistemas UNIX existen multitud de archivos de menos de 100 bytes de tamaño. En algunas instalaciones (news servers) muchos más

La utilización de un bloque de datos entero provoca fragmentación (perdida de espacio aprox. 6%)

Soluciones:

- Incluir parte de los archivos en el propio i-nodo (los primeros bytes): [e.g. XFS]
- Usar bloques de datos que compacten varios archivos (o finales de archivos mayores) [ReiserFS].

4) Fiabilidad del SA

Fallos HW y SW pueden provocar fallos en un volumen

Soluciones Hardware:

- Controladores que tratan con sectores con fallos
- Discos con información redundante (RAID)

Backups (copias de respaldo):

- En discos
- En cintas
- Backups incrementales

4) Fiabilidad del SA

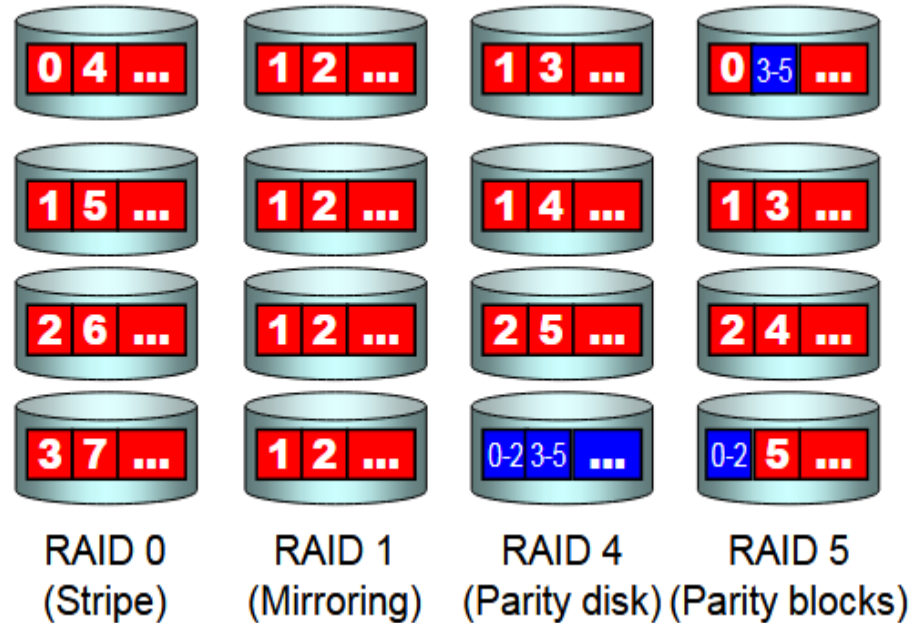
Sistema de discos redundantes RAID:

[0] Sin redundancia

[1] Copia de la información
(discos espejo)

[4] Un disco con paridades

[5] Los bloques de paridades se
distribuyen entre los discos



3) Consistencia del SA

Dos aspectos importantes:

- Comprobar que la estructura física del sistema de archivos es coherente
- Verificar que la estructura lógica del sistema de archivos es correcta

Estructura física: se comprueba la superficie del dispositivo de almacenamiento

Estructura lógica:

1. Se comprueba que el contenido del superbloque responde a las características del sistema de archivos
2. Se comprueba que los mapas de bits de i-nodos se corresponden con los i-nodos ocupados en el sistema de archivos
3. Se comprueba que los mapas de bits de bloques se corresponden con los bloques asignados a archivos
4. Se comprueba que ningún bloque esté asignado a más de un archivo
5. Se comprueba el sistema de directorios del sistema de archivos, para ver que un mismo i-nodo no está asignado a más de un directorio

3) Consistencia del SA

Consistencia de los archivos:

- Contador de enlaces > contador real
- Contador de enlaces < contador real
- Número de i-nodo > Número total de i-nodos
- Archivos con bits de protección 0007
- Archivos con privilegios en directorios de usuario

UNIX y LINUX: fsck

- fsck /dev/dsk/c0t0d3s1
- Comprueba el dispositivo físico c0t0d3s1

Consistencia del SA

Número de bloque						
0	1	2	3			n
1	0	1	0	...		0
Referencias						
0	1	2	3			n
0	1	0	1	...		1
Libres						

CASO 1

Correcto

Número de bloque						
0	1	2	3			n
1	0	2	0	...		0
Referencias						
0	1	2	3			n
0	1	0	1	...		1
Libres						

CASO 3

Éste es el peor caso

Número de bloque						
0	1	2	3			n
1	0	0	0	...		0
Referencias						
0	1	2	3			n
0	1	0	1	...		1
Libres						

CASO 2

Se deja libre

Número de bloque						
0	1	2	3			n
1	0	1	0	...		0
Referencias						
0	1	2	3			n
0	1	1	1	...		1
Libres						

CASO 4

Se deja ocupado

Journaling del SA

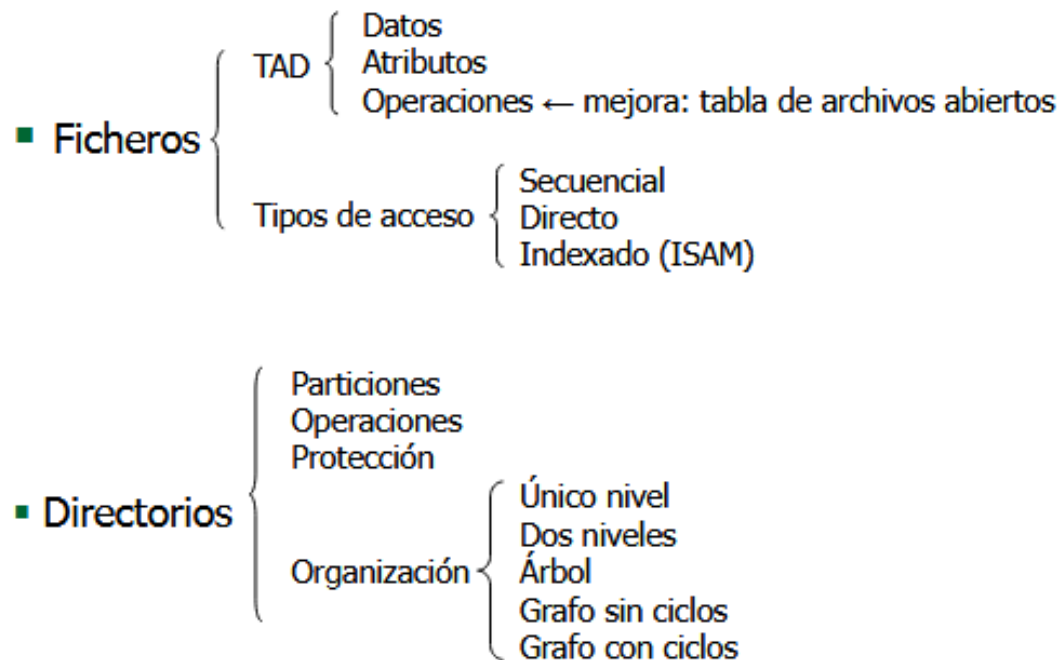
Para evitar la verificación (fsck) de sistemas de ficheros de gran tamaño, que sería muy costosa, se implementa un modelo de control transaccional basado en logging:

- Toda suboperación que modifique los metadatos y datos de un archivo se agrupa en la misma transacción
- Si el sistema falla, las acciones parcialmente realizadas se deshacen o completan recorriendo el log
- No se garantiza que el sistema esté actualizado al finalizar la recuperación, sino que es consistente.

Sistemas con esta filosofía son: JFS (IBM), ext3fs y la gran mayoría de sistemas de archivos modernos.

RESUMEN

Interfaz con el sistema de archivos



RESUMEN

Implementación del sistema de archivos

- Métodos de asignación
 - Contigua { Simple
Con extensiones
 - Enlazada { Simple
Por clusters
FAT
 - Indexada { Simple
Enlazado
Múltiples niveles
Combinado
- 1 ▪ Administración del espacio libre { Vector de bits)
Lista enlazada de bloques libres
Lista enlazada con info. de bloques libres
FAT ?
- Implementación de directorios { Lista lineal
Tabla de dispersión (hash)
Árbol B*